Vysoké učení technické v Brně
Fakulta informačních technologií

Síťové aplikace a správa sítí

Exportér SNMP Gauge metrik
do OpenTelemetry (OTEL)

17. 11. 2025 Smirnov Nikita (xsmirn02)

Obsah

1 Úvod **2**

2 Návrh **2**
2.1 Hlavní moduly, které tvoří jádro programu 2
2.2 Prostudováno před implementací 3
2.3 Příklad studované interakce 3

3 Implementace **4**
3.1 Workflow aplikace 5
3.2 Upřesnění implementace 5
3.2.1 Bod 1.4 5
3.2.2 Bod 4.2.1 6

4 Použití **7**

5 Testování **7**
5.1 Struktura unit testů 7
5.2 Spuštění testů 8
5.3 Příklad spuštění testů 8
5.4 Příklad integračních testů 9

1 Úvod

Tento projekt se zaměřuje na implementaci aplikace `snmp2otel`, která slouží jako exportér metrik ze zařízení spravovaných protokolem SNMPv2c do systému OpenTelemetry (OTEL). Cílem aplikace je periodicky získávat data pomocí dotazů SNMP, zpracovat přijaté odpovědi v kódování ASN.1 BER a exportovat je ve formátu OTLP/HTTP JSON na zadaný OTEL endpoint.

2 Návrh

Program `snmp2otel` byl navržen tak, aby bylo možné jej snadno rozšiřovat a podporovat moduly. Při vývoji byl také použit přístup TDD (test-driven development) k zajištění kontroly funkčnosti aplikace ve všech fázích a také pro spolehlivou a bezpečnou implementaci nových funkcí.

2.1 Hlavní moduly, které tvoří jádro programu

- **Konfigurační modul** Načítá parametry příkazové řádky, zpracovává soubory OID a mapování, validuje cílové adresy a porty. Umožňuje volitelné mapování OID na jména a jednotky. Ignoruje jiné typy než `gauge`.
- **Logovací modul** Umožňuje verbose a debug režimy, výpis hexdumpů, logování chybových stavů a pomocných informací.
- **ASN.1 BER Encoder / Decoder** Ručně implementované funkce pro práci s BER podle normy ITU-T X.690. Podporované typy: Integer, OctetString, ObjectIdentifier, Sequence, Null, GetRequest/Response PDU. Pro práci s metrikami je však podporován pouze gauge, vzhledem k tomu, že v zadání je uvedeno:
  Program musí podporovat pouze metriky typu Gauge.
  V případě potřeby jsou však všechny funkce umožňující zpracování a exportování typů, jako je `octstring`, `int`, `TimeTicks`, již v kódu k dispozici.
- **SNMP klient** Implementuje odeslání SNMP dotazu a zpracování odpovědi. Obsahuje ošetření timeoutů, poškozených paketů a chyb typu. Pokud se velikost uvedená v hlavním tagu neshoduje se skutečnou velikostí paketu, program se pokusí extrahovat co nejvíce informací. Přitom bude adekvátně a přesně reagovat na chyby při vytváření paketů.
- **OTEL Exporter** Ručně sestavuje JSON[1] podle OTEL specifikace a odesílá jej pomocí HTTP POST. Ošetřuje chyby serveru, výpadky spojení, prázdné odpovědi a chybové kódy.

[1] Původně byla myšlenka použít hotová řešení pro formátování JSON nebo napsat vlastní modul pro práci s ním. První myšlenka však byla zamítnuta, protože v dané implementaci byla závislost na nestandardních knihovnách snížena na nulu. Druhá myšlenka byla zamítnuta, protože práce s JSON není v daném projektu zásadní.

2.2 Prostudováno před implementací

Před zahájením práce byla provedena analýza stávajících standardů a nástrojů pro práci s metrikami SNMP a OTEL, díky čemuž byla následná implementace realizována v kratším čase a s menším počtem výzkumů. Nastudované standardy:

- RFC 1157 — protokol SNMP[SNM90]
- RFC 3416 — SNMPv2 PDU struktura[SNM02]
- ITU-T X.690 — ASN.1 BER kódování[x6921]
- OpenTelemetry OTLP/HTTP specifikace[ote24]

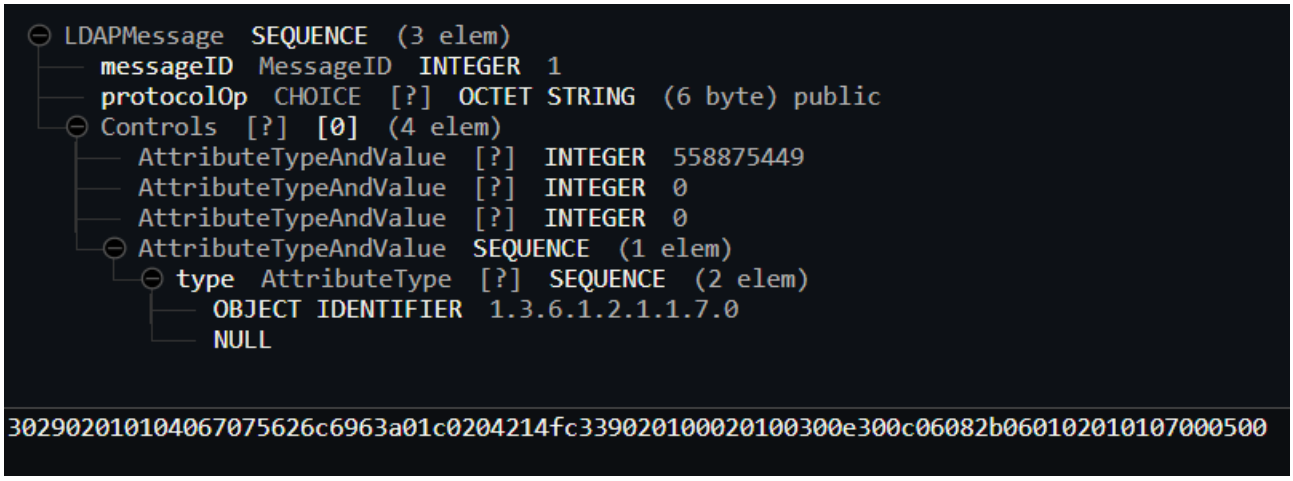
Před zahájením práce byla také provedena analýza práce SNMP, principu kódování dotazů a odpovědí, jakož i analýza různých potenciálních situací, které mohou nastat při komunikaci s agentem SNMP, pomocí těchto nástrojů:

- `snmp-sim` — simulace SNMP agentů[Sou23]
- `snmpget` — odesílání požadavků na SNMP agenta
- Wireshark — analýza UDP provozu[Wir24]
- ASN.1 Decoder — zobrazení BER struktur[Luc24]

2.3 Příklad studované interakce

snmpget - spuštěný pomocí:

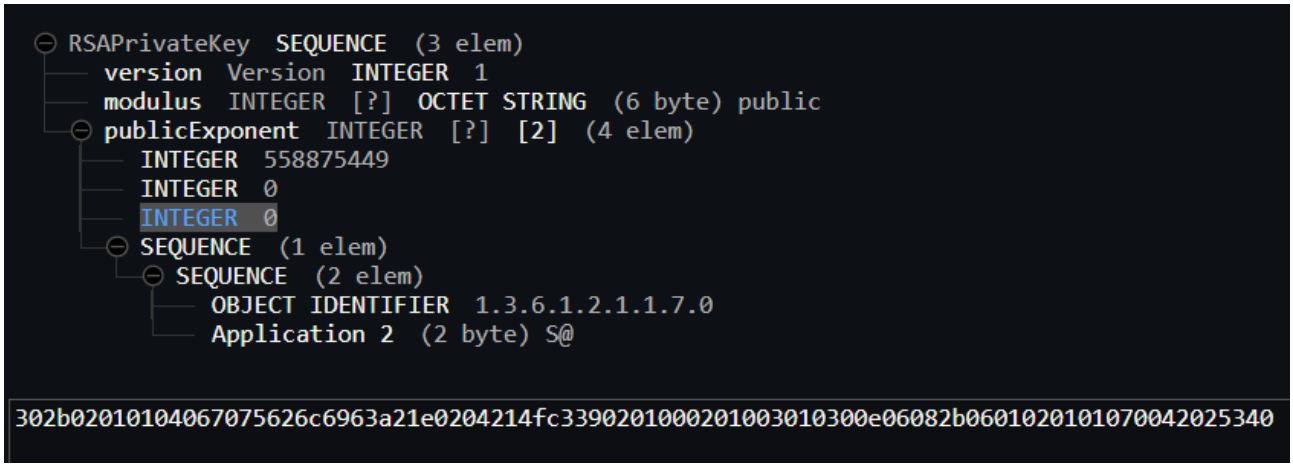
```
snmpget -v2c -c public 127.0.0.1:16100 1.3.6.1.2.1.1.7.0
```



Obrázek 1: Zachycený požadavek od `snmpget`.

snmp-sim - spuštěný pomocí:

```
snmpsim-command-responder --data-dir=./snmp_data
--agent-udpv4-endpoint=127.0.0.1:16100
```



Obrázek 2: Zachycená odpověď od `snmp-sim`.

Zde můžeme vidět, že zpráva je sestavena podle následujícího principu:

1. Hlavní sekvence, která zahrnuje celý balíček
2. Verze
3. community string
4. PDU
   - 4.1 Request-id
   - 4.2 Error
   - 4.3 Error-index
5. Varbinds
   - 5.1 Sekvence, která zahrnuje jeden varbind
   - 5.2 OID
   - 5.3 NULL - pro požadavek, Data - pro odpověď

3 Implementace

Jak již bylo zmíněno, implementace programu probíhala postupně v souladu s workflow. Před napsáním každého modulu byly také napsány testy, do kterých byla nejprve vložena očekávaná odezva daného modulu.

3.1 Workflow aplikace

1. Config.h
   - 1.1 Spustí se zpracování argumentů, kontrola hodnot a jejich typů.
   - 1.2 Na základě argumentů se inicializuje logger.
   - 1.3 Probíhá kontrola přítomnosti povinných argumentů.
   - 1.4 Probíhá parsování oidů a mapování.
2. snmp.h
   - 2.1 Probíhá počáteční inicializace modulu a otevření portu.
   - 2.2 Připojují se oidy, podle kterých bude prováděn dotaz agentu snmp.
3. otel.h
   - 3.1 Probíhá počáteční inicializace modulu.
   - 3.2 Probíhá načítání mapování pro následné použití.
4. Main cycle
   - 4.1 Probíhá dotaz na SNMP agenta.
     - 4.1.1 Balíček se sestaví a při následujících přístupech se použije předkompilovaná verze balíčku[2].
     - 4.1.2 Probíhá pokus o odeslání balíčku na agent snmp, v případě neúspěchu se program ukončí, protože agent snmp je považován za nedostupný. V tomto případě nemá smysl pokračovat v práci programu.
     - 4.1.3 V případě úspěchu program zpracuje obdrženou odpověď a exportuje ji ve formě struktury.
   - 4.2 Metriky se exportují na OTEL endpoint.
     - 4.2.1 Získané metriky jsou kódovány podle standardu OpenTelemetry.
     - 4.2.2 Program se pokouší připojit k serveru. Pokud server není platný nebo nepřijímá připojení, program se ukončí, protože další zpracování nemá smysl. V jiných případech, například pokud server neodpovídá na požadavky nebo odpovídá chybou, program o tom informuje v verbose režimu a pokračuje v práci.
   - 4.3 Program čeká na zadaný interval před opakováním cyklu.

3.2 Upřesnění implementace

3.2.1 Bod 1.4

Pro oidy je podporován formát uvedený v zadání, formát musí být striktně dodržen, v případě, že při čtení dojde k chybě, program se ukončí, stejně jako v případě, že nebyl načten žádný oid.

[2]Původně byl nápad používat předkompilované balíčky vždy, ale bohužel při každém požadavku je třeba zadat nové req-id, což vede k tomu, že je třeba balíček znovu zabalit do vnější sekvence.

```
#One OID per line
1.3.6.1.2.1.1.7.0
#You can leave comments with #
1.3.6.1.2.1.1.8.0

#Also you can leave empty lines
```

Pro mapování byl použit formát uvedený níže. Je však třeba zdůraznit několik věcí.

1. Podle zadání jsou podporovány pouze metriky Gauge, ostatní budou ignorovány.
2. Mapovací soubor není povinný.
3. Mapovací soubor může být prázdný nebo může obsahovat chyby, na rozdíl od OIDů však tato situace nezpůsobí chybu.
4. Není nutné zadávat mapování pro všechny oidy. Stejně tak je možné zadat mapování pro oidy, které se nenacházejí v souboru s oidy.
5. Je možné zadat mapování pro jeden oid několikrát, použije se poslední varianta.
6. Není nutné vyplňovat všechna pole. Pole však musí být vyplněna postupně.
7. Mezery a tabulátory jsou povoleny, při zpracování budou ořízнuty.
8. Mapovací soubor se hledá podle principu - `((.+)\.or(.+))` + .map
   - 8.1 OID soubor = oids.txt - Mapping soubor = oids.map
   - 8.2 OID soubor = oids - Mapping soubor = oids.map
   - 8.3 OID soubor = oids.test.txt - Mapping soubor = oids.test.map

```
#You can leave comments with #
#TYPE = {"gauge"}
#One mapping per line
#Address         |Name               |Unit   |TYPE
1.3.6.1.2.1.1.7.0|SomeRndNumber21312|ms     |gauge

#Also you can leave empty lines
1.3.6.1.2.1.1.8.0|SomeSrandom1234   |ns     |gauge
1.3.6.1.2.1.1.9.0|JustName
```

3.2.2 Bod 4.2.1

Načtené hodnoty jsou vloženy do struktury:

```
resourceMetrics:
  scopeMetrics:
    metrics:
      - name
        unit
        type: gauge
        datapoints:
          - asInt
```

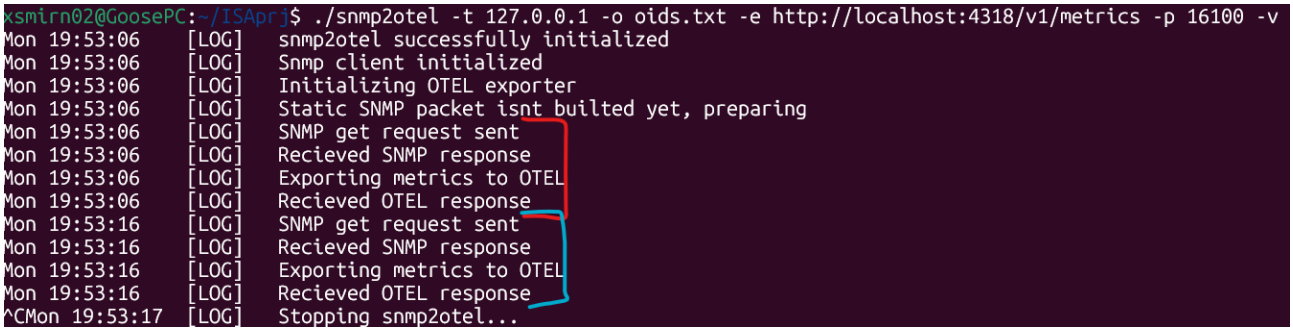
4 Použití

Aplikace se spouští následujícím způsobem:

```
./snmp2otel -t <target> -o <OIDs file> -e <OTEL endpoint> -p <port>
```

Ukázka:

```
./snmp2otel -t 127.0.0.1 -o oids.txt \
            -e http://localhost:4318/v1/metrics -p 16100
```



Obrázek 3: Příklad spuštění se dvěma cykly odesílání metrik a zapnutým verbose režimem

Tipy k argumentům získáte zadáním:

```
./snmp2otel
    or
./snmp2otel --help
```

5 Testování

V tomto projektu bylo testování věnováno mnoho času, díky čemuž se podařilo projekt rozšiřovat a rychle sledovat chyby, které se vyskytly při vývoji.

5.1 Struktura unit testů

Unit testy se spouštějí z hlavního souboru - `tester.cpp`. A rozděleny podle modulů do souborů:

1. `tester_config.cpp` - 21 test
2. `tester_ber.cpp` - 32 testů
3. `tester_snmp.cpp` - 7 testů
4. `tester_otel.cpp` - 9 testů

Používá se také pomocný soubor, který umožňuje sjednotit testování a výstup referenčních informací - `test_utils.cpp`.
Podle názvu souborů lze určit, ke kterému modulu se testování vztahuje. Lze také poznamenat, že v prvních modulech testování pokrývá mnohem větší objem kritických situací – to souvisí s tím, že první moduly jsou určeny k zachycení většiny chyb, které může uživatel udělat. Poslední moduly se setkávají s chybami hlavně kvůli problémům se serverem, na což je kladen důraz.

5.2 Spuštění testů

Obecná varianta

```
make test
```

Rychlé testy (s výjimkou testů, které vyžadují čekání. V současné době se jedná o testy s časovým limitem v SNMP).

```
make test-fast
```

Spuštění testů pod valgrindem za účelem kontroly úniků paměti v různých situacích

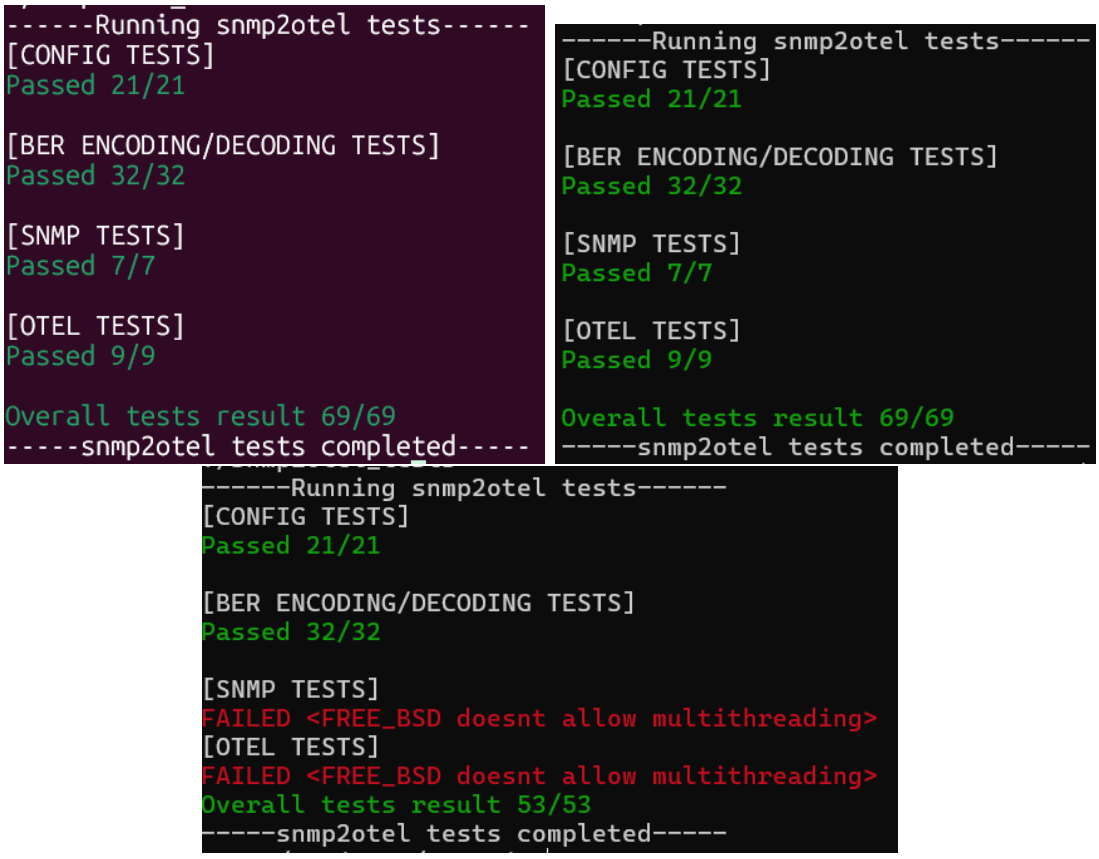
```
make valgrind-tests
```

Pro integrační testy lze použít falešné servery:

```
make snmp_integration
make otel_integration
```

5.3 Příklad spuštění testů

Zde je možné si všimnout, že na Eve neprobíhají všechny testy, ale tato vlastnost je dána pouze tím, že Eva nepodporuje spouštění více vláken, které jsou nutné pro fungování testů v SNMP a OTEL.



Obrázek 4: Příklad spuštění testů na různých platformách

Příklad spuštění testů pod valgrindem. Jak je patrné, v aplikaci nebyly zaznamenány žádné úniky, přičemž testy pokrývají jak situace běžného použití, tak situace s chybným chováním.

```
=== Running tests under Valgrind ===
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./snmp2otel_tests
==437633== Memcheck, a memory error detector
==437633== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==437633== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==437633== Command: ./snmp2otel_tests
==437633==
------Running snmp2otel tests------
[CONFIG TESTS]
Passed 21/21

Overall tests result 69/69
-----snmp2otel tests completed-----
==437633==
==437633== HEAP SUMMARY:
==437633==     in use at exit: 0 bytes in 0 blocks
==437633==   total heap usage: 10,492 allocs, 10,492 frees, 1,725,307 bytes allocated
==437633==
==437633== All heap blocks were freed -- no leaks are possible
==437633==
==437633== For lists of detected and suppressed errors, rerun with: -s
==437633== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Obrázek 5: Příklad spuštění testů pod valgrindem

5.4 Příklad integračních testů

Spuštění a průběh SNMP serveru

```
102010108004202040d2300d06082b06010201
SNMP stub listening on UDP 16100
(Press Ctrl+C to stop)
SNMP request received (71 bytes)
SNMP response sent (76 bytes)
^C
```

Obrázek 6: Zpracování jednoho požadavku SNMP

Spuštění a průběh OTEL serveru

```
./int_otel
OTEL stub started on 4318
(Press Ctrl+C to stop)
==== OTEL REQUEST ====
POST /v1/metrics HTTP/1.1
Host: localhost
Content-Type: application/json
Content-Length: 860
Connection: close

{
 "resourceMetrics": [
    {
        "scopeMetrics": [
            {
                "metrics":[
                        {
                            "name": "SomeRndNumber21312",
                            "unit": "ms",
                            "type": "gauge",
                            "gauge": {
                                "dataPoints": [
```

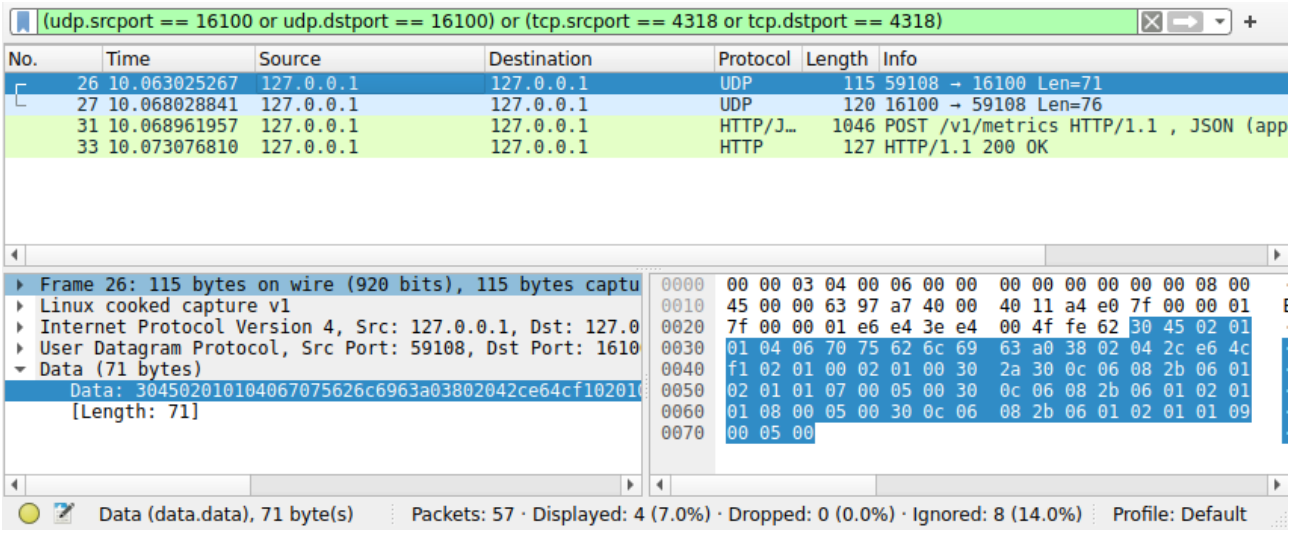
Obrázek 7: Zpracování jednoho požadavku OTEL

Provoz hlavního programu

```
xsmirn02@GoosePC:~/ISAprj$ ./snmp2otel -t 127.0.0.1 -o oids.txt -e http://localhost:4318/v1/metrics -p 16100 -v
Mon 20:59:49    [LOG]   snmp2otel successfully initialized
Mon 20:59:49    [LOG]   Snmp client initialized
Mon 20:59:49    [LOG]   Initializing OTEL exporter
Mon 20:59:49    [LOG]   Static SNMP packet isnt builted yet, preparing
Mon 20:59:49    [LOG]   SNMP get request sent
Mon 20:59:49    [LOG]   Recieved SNMP response
Mon 20:59:49    [LOG]   Exporting metrics to OTEL
Mon 20:59:49    [LOG]   Recieved OTEL response
^CMon 20:59:51  [LOG]   Stopping snmp2otel...
```

Obrázek 8: Zpracování jednoho požadavku snmp2otel

Zachycená komunikace



Obrázek 9: Zpracování jednoho požadavku

Reference

[Luc24] Lapo Luchini. Asn.1 javascript decoder. https://lapo.it/asn1js/, 2024.

[ote24] Opentelemetry protocol specification (otlp). https://opentelemetry.io/docs/specs/otlp/, 2024.

[SNM90] Rfc 1157 - simple network management protocol (snmp). https://datatracker.ietf.org/doc/html/rfc1157, 1990.

[SNM02] Rfc 3416 - snmpv2 message processing and pdu structure. https://datatracker.ietf.org/doc/html/rfc3416, 2002.

[Sou23] Open Source. Snmp simulator (snmp-sim). https://github.com/etingof/snmpsim, 2023.

[Wir24] Wireshark. Wireshark protocol analyzer. https://www.wireshark.org/, 2024.

[x6921] Itu-t x.690 - asn.1 encoding rules: Ber, cer, der. https://www.itu.int/rec/T-REC-X.690, 2021.